INTERVIEW DEFENSE DOCUMENT

PulseRank Platform v1.0

Production-simulated marketplace recommendation system with IPS bias correction, delayed attribution, exposure governance, offline A/B simulation, and evidence artifacts.

Metric	Value
Sessions	4,132
Items / Sellers	650 / 80
Impressions	41,320
Purchases	539
Hybrid Recall@100	0.690
Holdout NDCG@10 (naive, biased)	0.134
IPS-weighted NDCG@10	0.522
Seller Gini after rerank	0.582
Catalog coverage@10	0.226
Offline A/B decision	HOLD_SIMULATED
JSON evidence artifacts	33
Failure scenarios	15

Sidharth Kriplani | May 2026

Section 1 — Hard Interview Q&A

Q1 — The naive holdout NDCG@10 is 0.134 but IPS-weighted NDCG@10 is 0.522 — a 4x difference. How do you know IPS is correcting position bias and not simply inflating the metric?

IPS re-weights each impression by the inverse of its propensity: items shown at rank 1 have a high propensity to be clicked regardless of relevance, so their contribution is down-weighted. Items shown at rank 10 have low propensity, so their clicks are up-weighted as strong relevance signal. If the 4x gap were an inflation artefact, we would see IPS scores consistently above naive across all rank positions. Instead, propensity_by_rank_report.json shows the expected inverse relationship — high propensities at ranks 1-3 and low propensities at ranks 8-10. The IPS weights at low ranks amplify the signal from items that were clicked despite poor display position, which is precisely the correction needed. The clipped IPS variant (max weight = 5.0) is used to control variance — without clipping, a single rank-10 click would receive an outsized weight that dominates the metric estimate.

Q2 — You use temporal holdout evaluation. Why does a random train/test split fail for recommendation systems?

Random splitting leaks future labels into training. In a marketplace, a user's purchase history up to day 45 should not influence the model trained on days 1-30. A random 80/20 split can place a day-45 session in the training set and a day-30 session in the test set, meaning the model has effectively seen outcomes from the future. This is especially problematic for collaborative filtering signals: if user A bought item X on day 45, a random split may include that signal in training while evaluating on day-30 sessions that predate the purchase. The result is optimistically biased offline metrics that collapse in online evaluation. PulseRank uses a strict temporal split — the last 20% of sessions by timestamp form the holdout, and no training signal from that period contaminates the model.

Q3 — The seller Gini coefficient after reranking is 0.582. Is that good or bad, and how was the pre-rerank Gini determined?

A Gini of 0.582 indicates moderate seller concentration — the top sellers still capture a disproportionate share of impressions, but not as severely as the pre-rerank baseline. The pre-rerank Gini is computed in diversity_report.json from the raw ranked output before MMR and seller constraint application. The governance target is not Gini = 0.0 (perfect equality, which would mean random exposure and destroy relevance) but a Gini below the pre-rerank baseline, demonstrating that exposure constraints are doing work. In practice, marketplace operators set a Gini tolerance based on business policy — PulseRank treats the tolerance as a configurable constraint, not a fixed target. The current value of 0.582 reflects the constraint as implemented; a tighter constraint would lower Gini but also lower NDCG since the most relevant items tend to be concentrated with a few high-quality sellers.

Q4 — Why is the A/B decision HOLD_SIMULATED rather than LAUNCH? What specific metric delta would flip it to LAUNCH?

The offline A/B simulation compares control (current ranker) vs treatment (challenger with IPS evaluation) using holdout session replay. HOLD_SIMULATED means the treatment did not show sufficient lift over control on the primary metric (IPS NDCG@10) to meet the threshold for a launch recommendation. The HOLD threshold in the offline decision gate requires a minimum relative improvement — without knowing the

exact threshold configured for this simulation, the ab_simulation_results.json artifact records the exact delta and decision boundary. The SIMULATED suffix is explicit: offline A/B is not equivalent to live A/B testing. It does not capture novelty effects, position-based treatment differences, or real user feedback loops. HOLD_SIMULATED is the correct outcome for a system with no live traffic — it honestly represents the offline evidence without overclaiming.

Q5 — Your hybrid candidate generation uses popularity and content-based signals. What are the cold-start failure modes for each?

Popularity scorer fails on new items with zero purchase history — they have a popularity score of 0.0 and are ranked last regardless of quality. This is the standard popularity cold-start problem. Content-based scorer fails on new users with no interaction history — without a user feature vector there is no similarity to compute. It also fails on items with sparse or missing content features (images, descriptions). Hybrid merging does not solve either problem: it combines scores linearly, so a zero-popularity item still gets a low combined score unless the content-based component is very high. The 15 failure scenarios in failure_recovery_report.json explicitly include cold-start for both new items and new users, with recovery paths: boosting new items by injecting a synthetic popularity floor, and falling back to session-based (context-only) recommendation for new users.

Q6 — Delayed conversion attribution uses a 30-day window. What happens to purchases that occur after 30 days, and how does this affect metric validity?

Purchases after the 30-day window are treated as unattributed — they do not contribute to any session's label. This creates a right-censoring problem for long-consideration products: luxury items or high-cost purchases may have consideration cycles of 60-90 days. Attribution window analysis in conversion_attribution_report.json shows the fraction of purchases attributed at 1 day, 7 days, and 30 days. If a significant tail extends beyond 30 days, the evaluation underestimates true conversion quality. The 30-day window is a standard industry convention (e.g., Facebook Ads uses 28-day click attribution) that balances recency with capture rate. A shorter window (7 days) would produce cleaner attribution but miss valid conversions; a longer window would introduce more noise from non-causal purchases. This is an acknowledged limitation: the metric is valid for the attribution model used, not for all possible purchase intents.

Q7 — IPS assumes the propensity model is correctly specified. What biases remain if the position-based click model is mis-specified?

Three biases survive IPS correction if the propensity model is wrong. First, residual position bias: if the click model underestimates propensity at rank 1 (too easy to click), items at rank 1 are over-weighted after IPS correction, inflating their contribution. Second, presentation bias not captured by rank: IPS uses display_rank as the only confound. If click probability also depends on item image quality, description length, or price display — and these are not included in the propensity model — the IPS correction is incomplete. Third, selection bias in the impression set: users who see rank-10 items are a self-selected population (they scrolled past 9 items), which the position-based model does not account for. The clipping at max_weight = 5.0 partially mitigates amplification of propensity estimation errors but does not eliminate the bias from mis-specification.

Q8 — The catalog coverage@10 is 0.226. What does this mean and when does it matter?

Catalog coverage@10 is the fraction of all 650 items that appear in at least one top-10 recommendation list across all sessions. A value of 0.226 means only 22.6% of the catalog — about 147 items — ever receives a top-10 impression. The remaining 77.4% of items are effectively invisible to users in ranked recommendations. Coverage matters when the platform has a long-tail of items that deserve exposure: niche categories, new seller inventory, or items with high margin but moderate popularity. Low coverage concentrates purchases on already-popular items, creating a feedback loop that reinforces popularity bias. Coverage@10 = 0.226 is low for a 650-item catalog — a production system would target at least 60-70% coverage by injecting exploration slots. PulseRank reports it as an evidence metric, not a constraint, because coverage vs. relevance is a business policy decision.

Q9 — What does PulseRank NOT demonstrate that a production recommendation system requires?

Seven things. First, online A/B testing — there is no live traffic and no real user feedback loop. Second, real-time inference — the entire system runs batch, not in milliseconds at request time. Third, personalisation at scale — user feature vectors are synthetic; there is no user embedding trained on real click history. Fourth, multi-objective optimisation — the system optimises a single metric (IPS NDCG@10); production systems balance revenue, diversity, novelty, and policy constraints simultaneously. Fifth, bandit/RL exploration — there is no contextual bandit for exploration-exploitation tradeoff. Sixth, real inventory constraints — the system does not handle out-of-stock, geofencing, or eligibility rules. Seventh, A/B infrastructure — there is no feature flag system, traffic splitting, or experiment logging. These are explicitly documented in the README truth boundary.

Q10 — You report MetaSignal-compatible events. What are those and why are they in a ranking system?

The 15 structured events in metasignal_integration_events.json follow the MetaSignal event schema — a common event format designed for observability across multiple systems. Including them demonstrates that PulseRank is instrumented for production monitoring: key lifecycle events (model_version_deployed, ab_experiment_started, gini_constraint_violated, cold_start_fallback_triggered) are emitted as structured logs. In a real production system, these events would flow to a metrics store (e.g., MetaSignal) where they can trigger alerts, anomaly detection, or dashboard updates. The integration is simulated — no real event bus — but the schema compatibility means the events are directly ingestible by a MetaSignal-like monitoring layer. This is a deliberate portfolio connection: PulseRank produces signals that MetaSignal is designed to consume.

Section 2 — What PulseRank Does Not Claim

No live traffic. All evaluation is offline replay on synthetic data. No real users were served.

No real-time serving. Batch pipeline, not millisecond inference. No serving infrastructure.

No RL / contextual bandits. Exploration-exploitation is not implemented. Static offline evaluation only.

No online A/B testing. HOLD_SIMULATED is an offline decision. No live experiment infrastructure.

No real inventory constraints. No out-of-stock, geofencing, or eligibility rules applied.

No multi-objective RL reward. Single-metric optimisation. Production would balance revenue, diversity, and policy simultaneously.